APCS 程式設計系列 · U04 · 實作初級～中級 · C++版

一維陣列與前綴和 詳解

陣列宣告與走訪 · 索引從 0 開始與越界 · 計數陣列 · 前綴和 · 雙指標與滑動視窗 · 反轉與平移

Part A 答案速查

1	2	3	4	5	6	7	8	9	10
A	D	A	B	B	D	C	C	C	D
11	**12**	**13**							
B	A	A							

Part A 程式識讀詳解

第 1 題 答案：(A)

- i 依序是 0、2、4，取到 a[0]、a[2]、a[4]，也就是 3、4、5。
- 3 + 4 + 5 = 12。

【錯】把索引當成從 1 開始，會取到 1、1、9，得到 11。

第 2 題 答案：(D)

- 大括號只給部分初始值時，其餘格子自動補 0：a 是 {1, 2, 0, 0, 0}。
- s = 1×1 + 2×2 = 5。

【錯】完全沒有初始化的區域陣列（int a[5];）才是亂數；有寫大括號就會補 0。

第 3 題 答案：(A)

- sizeof(a) 是整個陣列的位元組數（6 × 4 = 24），除以一格的大小 4，得到元素個數 6。
- 最後一個元素是 a[n - 1] = a[5] = 42。

【錯】只寫 sizeof(a) 會得到 24（位元組數），不是元素個數。

第 4 題 答案：(B)

- c[v] 記錄數字 v 出現幾次：0 出現 1 次、1 出現 3 次、2 出現 0 次、3 出現 2 次、4 出現 2 次。
- 依序印出 1、3、0、2、2。

第 5 題 答案：(B)

- 計數後 c[3] = 3、c[7] = 3、c[2] = 2、c[1] = c[9] = 1。
- 第二個迴圈照「數值」從小掃到大，先遇到3；7的次數沒有「大於」3的次數，不會換掉。印出33。

【錯】7 在陣列裡比較早出現，但找最多的迴圈是照數值大小掃，不是照出現順序。

第 6 題 答案：(D)

- p 依序是 0、5、7、14、15、21。
- p[4] - p[1] = 15 - 5 = 10，也就是 a[2] + a[3] + a[4] = 2 + 7 + 1。

【錯】p[4] - p[1] 是「第 2 到第 4 個」的和，第 1 個已經被扣掉了。

第 7 題 答案：(C)

- 每一格加上「已經更新過」的前一格，a 變成 2、6、7、10、15（原地做前綴和）。
- a[2] = 7，a[4] = 15。

【錯】a[i - 1] 是更新後的值，不是原本的值。

第 8 題 答案：(C)

- 每次從 l 加到 r 是 O(n × q)，最多約 4×10^{10} 次，超時。
- 預先算所有 (l, r) 要 $O(n^2)$，時間和記憶體都不夠；排序會打亂順序，根本不對。
- 前綴和：建表 O(n)、每次詢問 O(1)，總共 O(n + q)。

第 9 題 答案：(C)

- 這是長度 3 的滑動視窗，五個視窗的和依序是 7、6、7、8、9。
- 和 >= 7 的有 7、7、8、9，共 4 個。

【錯】別忘了第一個視窗（和為 7）在迴圈外面已經算過一次。

第 10 題 答案：(D)

- 1 + 9 = 10，c = 1，兩邊往內；2 + 8 = 10，c = 2，兩邊往內。
- 5 + 6 = 11 太大，j 往左，i 和 j 相遇，結束。共 2 組。

第 11 題 答案：(B)

- i = 0、1 時把陣列換成 5 4 3 2 1；i = 2 自己跟自己換。
- i = 3、4 又把同樣兩對換回來，結果回到 1 2 3 4 5。

【錯】反轉只能走到中間（i < n / 2），走完全部等於換了兩次。

第 12 題 答案：(A)

- i = 1：a[1] = a[0] = 1；i = 2：a[2] = a[1]，但 a[1] 已經被改成 1 了。
- 一路覆蓋下去，全部變成 1。

【錯】要得到 1 1 2 3 4，必須由右往左做（i 從 4 到 1）。

第 13 題 答案：(A)

- a[i] 被放到 b 的第 (i + 2) % 5 格：10 → b[2]、20 → b[3]、30 → b[4]、40 → b[0]、50 → b[1]。
- 這是「往右平移 2 格」，b 是 40 50 10 20 30。

【錯】b[(i + 2) % 5] = a[i] 是往右移；b[i] = a[(i + 2) % 5] 才是往左移（會得到 30 40 50 10 20）。

Part B 程式實作參考解答

實作 1 重複的號碼

解題思路

- 號碼範圍只到 10^6，開一個計數陣列 cnt，讀到 x 就 cnt[x]++。
- 最後 v 從 1 掃到 10^6：cnt[v] > 0 就是一種號碼，cnt[v] >= 2 就是重複的號碼。由小掃到大，輸出自然由小到大。

【錯】號碼最大是 10^6，計數陣列要開 10^6 + 1 格，cnt[1000000] 才不會越界。
【錯】兩層迴圈互相比較是 $O(n^2)$，n = 2×10^5 會超時。
【錯】一百萬格的陣列不要宣告在 main 裡（可能當掉），用 vector 或宣告成全域。

C++ 重複的號碼

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> cnt(1000001, 0);
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        cnt[x]++;
    }
    int kinds = 0;
    vector<int> rep;
    for (int v = 1; v <= 1000000; v++) {
        if (cnt[v] > 0) kinds++;
        if (cnt[v] >= 2) rep.push_back(v);
    }
    cout << kinds << "\n";
    if (rep.empty()) {
        cout << "none\n";
        return 0;
    }
    for (size_t i = 0; i < rep.size(); i++)
        cout << rep[i] << (i + 1 < rep.size() ? " " : "\n");
}
```

時間複雜度：O(n + M)，M = 10^6 是號碼範圍

實作 2 區間盈虧查詢

解題思路

- 建前綴和：p[0] = 0，p[i] = p[i-1] + a_i。
- 第 l 天到第 r 天的總和 = p[r] - p[l-1]，每個問題 O(1) 回答。

【錯】總和最多 $2\times10^5 \times 10^9 = 2\times10^{14}$，p 要用 long long。

【錯】每個問題都用迴圈從 l 加到 r 是 O(n × q)，會超時。

【錯】減的是 p[l-1] 不是 p[l]；l = 1 時要用到 p[0] = 0。

【錯】輸入輸出量大，要加 ios::sync_with_stdio(false); cin.tie(nullptr); 並用 "\n" 換行。

C++ 區間盈虧查詢

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> p(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        p[i] = p[i - 1] + x;
    }
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << p[r] - p[l - 1] << "\n";
    }
}
```

時間複雜度：O(n + q)

實作 3 連續 k 天最佳業績

解題思路

- 先算第 1 天到第 k 天的和，當作目前最大值，開始天數是 1。
- 視窗往右滑一格：加上新進來的 a[i]、扣掉離開的 a[i-k]。新的和「大於」目前最大值才更新，這樣平手時會留下最早的一段。

【錯】最大值從 0 開始：全部是負數時會錯（範例 2）。要用第一個視窗當初始值。
【錯】更新條件用 >，不要用 >=，否則平手時會換成比較晚的那一段。
【錯】k 天的和最多 2×10^{14}，要用 long long。
【錯】每個起點都重新加 k 個數是 O(n × k)，k 很大時會超時。

C++ 連續 k 天最佳業績

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<long long> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    long long s = 0;
    for (int i = 0; i < k; i++) s += a[i];
    long long best = s;
    int start = 1;
    for (int i = k; i < n; i++) {
        s += a[i] - a[i - k];
        if (s > best) {
            best = s;
            start = i - k + 2;    // 視窗是 a[i-k+1..i]，換成從 1 數
        }
    }
    cout << best << " " << start << "\n";
}
```

時間複雜度：O(n)

實作 4 兩數湊目標

解題思路

- 陣列已經排好序，用雙指標：i 從最左邊、j 從最右邊。
- 和等於 T：計數一次，i、j 同時往內（數字互不相同，a_i 只能配這一個 a_j）。
- 和太小 i 往右；和太大 j 往左。i 和 j 相遇就停，最多走 n 步。

【錯】$a_i + a_j$ 最大接近 2×10^9，超過 int 的上限（約 2.1×10^9）。a 和 T 要用 long long，否則相加會溢位。

【錯】兩層迴圈試所有 (i, j) 是 $O(n^2)$，$n = 2\times10^5$ 會超時。

【錯】條件是 i < j，同一個數不能自己配自己（例如 T = 4 時 2 + 2 不算）。

C++ 兩數湊目標

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long t;
    cin >> n >> t;
    vector<long long> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    int i = 0, j = n - 1;
    long long cnt = 0;
    while (i < j) {
        long long s = a[i] + a[j];
        if (s == t) {
            cnt++;
            i++;
            j--;
        } else if (s < t) {
            i++;
        } else {
            j--;
        }
    }
    cout << cnt << "\n";
}
```

時間複雜度：O(n)

實作 5 旋轉輪盤

解題思路

- 不用真的搬資料。用一個變數 s 記錄「總共向左平移了幾格」：L k 就加 k，R k 就減 k，隨時對 n 取餘數。
- 最後一次輸出：新的第 i 格（從 0 數）放的是原本的 a[(i + s) % n]。

【錯】每個指令都真的搬 k 次，最多 $10^5 \times 10^9$ 次，一定超時；k 要先 % n。

【錯】C++ 的負數取餘數會是負的（-2 % 5 是 -2），往右平移要寫成 (s - k % n + n) % n。

【錯】分清楚方向：向左平移 s 格，新第 i 格 = 舊的第 i + s 格；寫反了就變成向右。

C++ 旋轉輪盤

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    long long s = 0;                    // 總共向左平移幾格
    while (m--) {
        char d;
        long long k;
        cin >> d >> k;
        k %= n;
        if (d == 'L') s = (s + k) % n;
        else s = (s - k + n) % n;       // 避免負數
    }
    for (int i = 0; i < n; i++)
        cout << a[(i + s) % n] << (i + 1 < n ? " " : "\n");
}
```

時間複雜度：O(n + m)

自我診斷表

對完答案，數一數每個觀念錯幾題：全對圈綠、錯 1 題圈黃、錯 2 題以上圈紅。紅燈的觀念回去看講義那一段。

觀念	題號	錯幾題	燈號（圈一個）	回去看哪裡
陣列走訪與索引	A1、A2、A3		綠 黃 紅	講義第 1、2 段（索引從 0 開始、越界）
計數陣列	A4、A5、B1		綠 黃 紅	講義第 3 段
前綴和	A6、A7、A8、B2		綠 黃 紅	講義第 4 段
雙指標與滑動視窗	A9、A10、B3、B4		綠 黃 紅	講義第 5 段
反轉與平移	A11、A12、A13、B5		綠 黃 紅	講義第 6 段